

Waltzing with Changes

Amy Law and Shawn Learn

TransCanada

450 – 1st Street SW

Calgary, Alberta

T2P 4K5, Canada

amy_law@transcanada.com and shawn_learn@transcanada.com

Abstract

*“The only thing that is a constant is change.”
Greek philosopher, Heraclitus*

This idiom is especially true when dealing with a software development project. With varying degrees, change impacts project delivery, budget, and quality. Often in today’s environment, the requirements for higher quality software combined with compressed timelines and fewer developers can make change management a difficult task. This paper focuses on how agile practices play a role in managing changes in an industrial software project developed at TransCanada. We discuss how the impacts of changes in resources, changes in requirements, and external factors can be minimized. We examine numerous methods of overcoming challenges that often occur during industrial software development.

1. Introduction

TransCanada is a leading North American energy company. The TransCanada network consists of approximately 41,000 kilometers (25,600 miles) of pipeline, which transports most of Western Canada’s natural gas production to growing markets in Canada and the United States. An average throughput of natural gas is about eleven billions cubic feet per day. TransCanada also owns, operates, and is developing more than 5,700 megawatts of power generation capacity. This would meet the needs of about 5.7 millions average households. To support this enterprise, the IS department has delivered many software solutions for specific business needs. One of

the software solutions developed in TransCanada is Project X¹.

Similar to other software development projects, Project X encountered various challenges. Requirement changes combined with fewer programmers and unexpected external influence made project success more elusive than ever. The project team needed to be creative to find solutions to overcome these challenges. Through a retrospective session, this paper examines how agile practices were tailored to manage changes and discusses numerous methods of overcoming challenges that often occur during industrial software development.

2. TransCanada Project

Table 1: Summary of Project X

	Project X
Type	Enhancement
Number of Programmers	6
Programming Skills	Intermediate
Programmer Dedication	Time Shared
Technologies	Java, Crystal Report, and Forte
Customers	Various Business Units
Duration	8 months
Original Delivery Schedule	December 2004
Actual Delivery Schedule	February 2005

As shown in Table 1, the goal of Project X was to enhance an existing Java web-enabled enterprise application. This enhancement was developed by six programmers with intermediate programming skills. The programmers were not exclusively working on

¹ Project X is a software development project in TransCanada, and its real name is not revealed due to confidentiality.

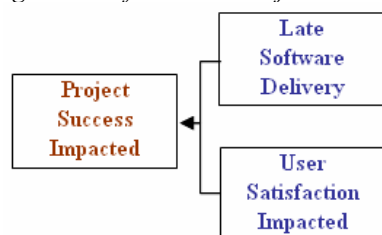
this project. They shared their time between various projects and application support during the course of software development. The end product was deployed to production in February 2005 serving various in-house business units and external customers.

A typical TransCanada software development project has two champions. Project X had a business champion representing the customer community, and an IS project leader representing the programming team. Both of them had previous exposure to agile methods and believed in the benefits of agile practices. They valued agile practices as an inexpensive way to manage the project, keep the business informed, and guide programmers to understand user requirements. Their belief was supported by TransCanada's IS department which delivered many software systems by adopting agile methods in the past five years. Leveraging from these lessons learned, Project X champions decided on adopting many agile practices. Some of the practices were master backlog, sprint backlog, daily status meeting, monthly scrum demo review, and small releases. These practices proved to be effective and beneficial when unexpected changes occurred during the course of project execution.

3. Unexpected Changes

Project X went through various challenges and unexpected changes during the course of software development. These challenges are typical in any industrial project. The changes affected project success as shown in Figure 1.

Figure 1: Influence to Project Success

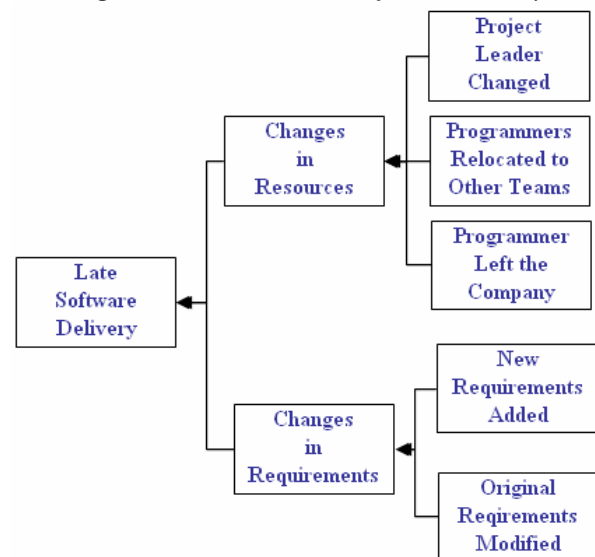


The project underwent several staffing changes. The manager was changed during the initial project evaluation. This had an immediate impact to the software delivery because it took a couple of months to hire and transition to a new manager. Several weeks later, two of the programmers originally allocated to Project X were pulled away to work on other projects with higher priority. A new resource was later identified to replace the two programmers. This new resource resigned for a career move a month before project completion. In the end, a new expert resource

was employed to complete the rest of the implementation. The limited resources and unforeseen staffing changes contributed to late software delivery.

In addition, Project X received system requirements changes midway through the project. Some of the new system requirements included a new security group to enable a larger user base and new report requirements for the ease of data verification. Other changes in the requirements were additional fields to existing screens and user interface modification to reduce navigation. Figure 2 shows how these changes contributed to late software delivery.

Figure 2: Causes to Late Software Delivery

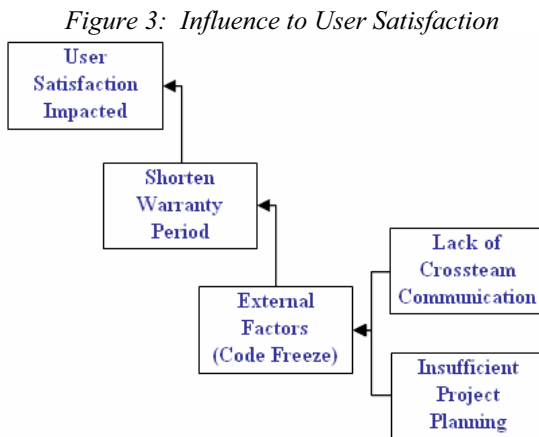


User satisfaction was also impacted by external factors. Project X initially planned a three-week warranty period². In the final month of the project, the team encountered an unexpected code freeze introduced by Project Y³. During the code freeze, programmers were not allowed to change source code unless the issue was a critical break fix. With the unexpected code freeze, the warranty period was reduced to one-week. The code freeze was not communicated to Project X until a couple of weeks prior to the actual freeze. A combination of inadequate cross team communication, often inherent in a large organization, and insufficient project planning

² Warranty period is a period of time when programmers fix any post-production issues, regardless of its nature major or minor, to ensure customer satisfaction.

³ Project Y is another software development project in TransCanada, and its real name is not revealed due to confidentiality.

shortened the warranty period, which impacted the overall user satisfaction as shown in Figure 3.



4. Analysis of Situation

In response to these changes, the project team had to analyze the situation to overcome the challenges in order to take proper corrective actions.

The resource utilization strategy in TransCanada is designed to maximize efficiency. Everyone in the team was occupied with projects, operational support, or both. In order to increase team efficiency, some of the project manager's ability to cope with unexpected workload needs to be sacrificed. Another source of inefficiency is the fact that programmers have obligations to many different business groups, all of whom have varied expectations. Satisfying the interests of one group may alienate others. This is a result of tradeoff between efficiency and flexibility [3].

With the limited resources, the project team had to find creative solutions to allocate the right people at the right time. Hiring new staff was a potential solution, but the project had a limited budget and limited time to locate programmers with the required technological skills. Hiring additional staff was not a viable solution.

As Brooks' Law indicates, adding manpower to a late software project makes it later. Although there are great advantages from splitting and assigning work to new programmers, the cost associated with the complexity on division of labor, transition, knowledge sharing, thorough communications, coordination and merging their work is a lot greater.

The project leader was new to the business functions and had limited project management experience. She had to learn the application requirements, and gain a deeper understanding of

business needs. This made communication of requirements difficult.

The in-house users and programmers from Project X and Project Y were located on different floors. The office layout did not facilitate convenient communication. The lack of communication between teams contributed to the unexpected code freeze. The stakeholders did not have enough time to react properly. As the code freeze was discovered in the final stage of the project, the programming team worked overtime and business accepted a shortened warranty period to overcome this problem.

5. Solutions

The project team considered a number of preventive controls and potential solutions to address these changes. Some of these controls and solutions leverage agile practices, whereas others are possible additions to the domain of agile methodology.

5.1. Solutions to Changes in Resources

Common problems in software development projects are insufficient experienced programmers and inefficient, ineffective use of their time. With constant change of resources, each new staff has to learn the system requirements and team's standard.

Project X attempted to adopt pair programming to minimize the impact of this problem. Pair programming allows the new staff to gain a better understanding of the system requirements, coding standard, and software design patterns from their peers. This enabled ongoing knowledge sharing and peer communication.

However, pair programming was difficult due to task switching. Programmers, with limited time, shared between numerous projects. This excessive multi-tasking jeopardized the benefits of pair programming. Therefore, pair programming was not applied consistently in Project X.

As a supplement, Project X applied code inspection. This practice complementarily used to ensure software quality. Extremely diligent documentation was adopted to minimize the impact of inconsistent pair programming. At the beginning, the project manager introduced a new Wiki web site to the team. She aimed to effectively create, centralize, and maintain documentation that facilitated the foundation of common understanding in system requirements. The Wiki web site was mainly used as a communication tool to clarify requirements. Only about 15% of the project time was invested to documentation.

One of the biggest challenges was to provide accurate estimates on project delivery. The original estimate accounted for pair programming which was not fully utilized. This resulted in a poor time and cost estimate. Several solutions could help to overcome this challenge. First, the monthly scrum demo review made the project progress transparent to the customer communities. Through monthly review, the project leader shared the updated time and cost estimates with the stakeholders. She also explained any deviation in order to manage expectations. This iteration exercise enabled the team to communicate, plan, forecast, and prioritize requirements for the work in the subsequent sprint.

The lack of resources caused late software delivery. To reduce this impact, small releases were deployed on a monthly basis. This allowed users to be aware of the software development progress. This also helped to manage expectations.

Another practice adopted was a formal change process. This process effectively and accurately communicated the changes to various levels. A project change control log was used to keep an up-to-date record of all changes, approved or otherwise. Specific trigger points determined when formal change process was necessary. Due to the change of resources, Project X had one formal change order; triggered by the delay of schedule. As shown in Table 2, these trigger points were developed in collaboration with business unit and project manager.

Table 2: Project Change Control Triggers

Change	Level	Trigger Details
Cost	Low	< \$ 10 K
	Med	\$ 10 K - \$ 15 K
	High	> \$ 15 K
Schedule: Intermediate Milestones	Low	< 1 Sprint change
	Med	> 1 Sprint change
	High	> 2 Sprints change
Schedule: Final Delivery	High	> 1 month delay
Product	Low	Changes with no benefits impact
	High	Changes with benefits impact

Formal change documentation of budget, delivery, and requirements ensured key stakeholders were aware of the project status and influencing factors. The sprint backlog and small releases provided justification

for changes. The decision was made with the business considering the priority of the project. Depending on trigger levels, different key stakeholders, such as business sponsor, champion, and project manager, must provide their signatures to approve or reject the changes. In this fashion, the formal change documentation was a communication tool, confirmation acknowledgement, and logging reference.

By adopting these practices, the business champion increased the ability to prioritize and the IS project leader was able to schedule work. Both leaders used these practices to expand or scale down the project.

5.2. Solutions to Changes in Requirements

A standard practice at TransCanada is to develop a project statement. This project statement must be signed prior to the start date of a project. It includes a high level of system requirements to define a project scope. In order to be flexible, the project leader has to respond properly to any requirement changes and additions during the project execution.

To ensure effective communication and thorough understanding between business users and programming team, Project X used a master backlog and sprint backlog to track the current system requirements, estimates, and velocity. The business champion prioritized requirements. The backlogs were used to show the velocity during the resource negotiation.

Another agile practice facilitated the changes in requirements was the tailored version of daily status meeting. Instead of having every stakeholder at the meeting, the business and IS side of the project had representatives who met on a regular basis. These representatives also paid informal visits to their team to obtain up-to-date progress and requirement changes. This allowed for the key benefit of the daily scrum meeting with a more efficient use of time.

Each individual was multi-tasking. Moving a dedicated staff to multiple projects could lead to a loss of at least six hours per week of that person's work time [3]. This hidden penalty could increase when the partitioning was greater [6]. The time-sharing nature also made it a challenge to justify the co-location moving costs⁴.

The use of conference calls was to minimize the problems associated with working at different locations and context switching. Due to the multitasking nature, the users and programmers did not make use of a standard scrum meeting on a daily basis.

⁴ A typical moving cost in TransCanada is C\$500.00 per person.

Instead, they had numerous conference calls per day. This allowed quick decision making whenever an issue would arise and prevented incorrect assumptions. For larger, more important issues, face-to-face meetings were held. It is important to note that it was not uncommon for the daily meetings to be cancelled. In contrast, some days multiple meetings were held. The flexible communication was the key to overcome the time sharing nature.

5.3. Solutions to Impact of External Factors

Despite the fact that resources and requirements were in continuous flux, the most influential change was the impact of external factors. Daily scrum meetings helped to alleviate many problems, but Project X developers and its key users were the only attendees. External forces played a significant role in the project where other project team did not participate the daily status meeting. Since programmers from these projects did not share their status, problems were not detected until late in the development cycle.

As another impact of multi-tasking, Project X's IS project leader was also the scrum master, system analyst, programmer, and tester. This overloading assignment increased the probability of insufficient overseeing. The result was that the code freeze was not detected, communicated, and consulted to the business unit until a couple of weeks prior to the freeze.

In response, the programmers worked overtime to cover the key features prior to the freeze and the business users worked overtime to perform user acceptance tests in a reduced elapsed time.

This is a contradiction to agile concept where sustainable pace is encouraged and to XP practice which uses a 40-hour workweek. Although agile practices are very useful guides, sometimes, alternative methods needed to be considered.

Daily scrum meetings gave an opportunity for everyone to share comments about the product and discuss any issues, particularly external factors that may affect the project. In this instance, the code freeze was missed because Project X and Project Y held isolated daily meetings. Therefore, it is extremely important to include the key individuals in the meetings and review the attendee lists on a regular basis.

The small releases, ranged from 30-days to 45-days, allowed other parts of the company to see this project impact on them. This is an important factor for expectation management. A combination of these tailored practices facilitated effective communication.

6. Reflection

Through a retrospective session, Project X team identified a number of practices that worked and several practices that did not. Finally, a list of lessons learned was documented to contribute to the IS organizational knowledge base.

6.1. Effective Practices

The project X team identified a number of practices that are worthwhile for future projects:

- It was highly effective having business leaders with power in the monthly review meeting to make quick binding decision.
- Business leaders with a strong engineering background and business acumen provided thorough user requirements, analogies, and explanations to the programming team.
- Sprint backlog was helpful to keep track of current and new requirements (color-coded). It measured the true velocity, predicted product delivery, and gave justification for formal change order.
- Monthly scrum demo review provided talking points for users to collaboratively discuss and finalize system requirements prior to deployment.
- Small releases made users aware of the realistic project progress. Flexible releases, ranged from 30-days to 45-days, proved to be effective in the small project.
- Convening pride at work, programmers were willing to work overtime to finish all of the key features before the code freeze.
- Users understood the importance of the code freeze and performed user acceptance tests in the reduced elapsed time.
- Users understood the importance of the code freeze and accepted the shortened warranty period.
- Open communication was consistently applied in the immediate team. Changes in resources and requirements were communicated early to ensure everyone could respond accordingly.
- Most staff was receptive to new practices, like agile methods, and they were willing to apply them to the project.
- It was important to consult users about the preference of delivery methodology and their availability for participations.

6.2. Unsuccessful Practices

Although agile methods were very beneficial, some of them required tuning based on the unique project environment:

- In some cases, pair programming was not and could not be consistently applied. Pair programming when needed only provided partial benefits of knowledge sharing.
- Daily scrum meeting was modified to accommodate the multi-tasking nature of the users and programmers. This did not provide the full benefits to overcome typical problems such as the lack of ongoing communication between teams.
- To some extent, the conference calls and frequent visits between floors facilitated communication. However, they did not always encourage the users and programmers to work closely together. There was room for improvement in co-location.
- Although there was a benefit to moving people to the same floor, the cost could be hard to justify. Alternatively, the company could set up a temporary station so that the common area could facilitate open communication.
- The intent of the monthly review was to provide communication opportunities to stakeholders. However, not all stakeholders attended the review. Some people were not included on the invitation list even though they might initiate changes affecting project. Not all of the stakeholders that were invited attended the review, due to their workloads and other responsibilities. This was a sign of insufficient communication between project teams. To ensure proper communication among various departments, it was critical to identify the stakeholders at the initial stage and review the stakeholder list on a regular basis. This was also a result of multi-tasking. The overloading assignment increased the probability of poor overseeing.
- Stable resource allocation could not be relied upon. A resource backup plan should have been developed to alleviate this instability.
- Programmers provided the original estimates might not be the developers involved in the project. Although the assumption was that the people doing the work were the ones who should be estimating, the constant changes voided this assumption.

- A single estimate with a deviation should be done. Monte Carlo estimation could be used to provide this more robust estimate. As an alternative, the estimates should be presented as a range such that the longer a unit task is the greater the range the estimate will need to be [7]. For example, the cost of a six-month project may have a range of $\pm 25\%$, whereas the cost of a one-month project may have a range of $\pm 5\%$.
- Estimation review process and feedback session with business leaders should take place. In this fashion, the programmers are responsible for the estimates, and the business units have the right to tune or withdraw the requirements once they realize the effort.

6.3. Lessons Learned

The retrospective wrap up session gave us a list of lessons learned that contributed to the IS organizational knowledge base:

- It was vital to build a high-level project scope with enough details to minimize requirement changes or additions.
- The team should revisit priority to ensure the high priority items would receive the most attention and resources.
- Velocity needed to be realistic and was an important tool to justify change order on timeline and cost.
- To explore better and cheaper options, it is advisable to educate users about the technological limitations and how their decisions influence the system requirements. The business could make a wiser decision and provide an alternative requirement easing the existing limitation.
- It was important to keep close radar on departments that could introduce changes to the project. Initially, the project leader should document stakeholders and their roles in a RACI matrix. RACI stands for Responsible, Accountable, Consulted, and Informed. The RACI matrix is a tool to assign and document task responsibilities.
- Another important tool was the risk registry, which documented the effect of changes and mitigation plan when such events occurred.
- The project leader should apply preventive controls on resource planning. At the beginning of the project, she should introduce a risk analysis of staff redeployment. This

risk analysis should determine the costs and associated impact of moving a staff away from the project. A sweet spot should be determined to ensure the optimal resource level is maintained.

- Another recommendation was to fill in the project team skills and expertise matrix to determine the strength of the team and consequence of staff changes.
- The project leader should also create or obtain a resource inventory list to identify resource expertise in the company.
- The project leader should create and maintain a backup staffing plan. This planning identifies the availability and costs of backup resources. The resources might not stay with the project for the entire cycle. Instead, they attend monthly review for consultation. This can be considered as an alternative to pair programming and used to overcome resource problem and project delay.
- It is important to implement a work management process where a backlog of required tasks are recorded, prioritized, measured, and analyzed. The work management process can help to facilitate a resource strategy.
- The company should provide timely training to staff to ensure adequate skill sets are available in the immediate workforce.
- The company should understand the multiple tasking penalties and prevent staff time-sharing.
- The company should embrace lessons-learned sessions to improve process and practices. The lessons learned should be communicated to business and IS departments. Therefore, future projects could reuse good practices and improve processes to fit in the unique environment.

7. Conclusion

As the famous Greek philosopher, Heraclitus challenged the world that “Everything is in flux.” It is common to find change in today’s software development projects. As shown in our industrial case, changes can be influential to project success, software

delivery, and user satisfaction. There are many ways to manage changes, but the most important aspect is communication. Communication needs to occur at many levels. Not only do the business and IS teams need to communicate between each other, but they also need to communicate within their own teams and with other groups that may influence the project. Although agile practices allow flexibility and prepare for changes, it is important to minimize the number of changes by practicing good effective communication early in the project. By applying and adopting the set of practices defined above, we believe that each of us can deliver successful software systems, provide effective business solutions, and increase customer satisfaction in our IS industry.

8. Acknowledgements

We would like to thank Rick Coutts, Jim Nicholas, Claudio Perrella, Shane Savard, Rebecca Wirfs-Brock, Scott Ambler, Chris Teare, Patrick Yip, Tam Nguyen, Dave Chiste, Raylene Charron, and Jackie Gardner, for their comments, guidelines, and encouragement.

9. References

- [1] Kent Beck, *Extreme Programming Explained – Embrace Change*, Addison Wesley Professional, 2000.
- [2] The Standish Group, “The CHAOS Report,” 1994.
http://www.standishgroup.com/sample_research/chaos_1994_1.php
- [3] DeMarco, Tom, *Slack*, Broadway, 2001.
- [4] Brooks, Frederick, *The Mythical Man-Month - Essays on Software Engineering*, Addison-Wesley Publishing Company, 1995.
- [5] ThinkReliability, *Cause Mapping – Effective Root Cause Analysis*, Houston, 2004.
- [6] Santosus, Megan, “Why More is Less,” 2003.
<http://www.cio.com/archive/091503/reality.html>
- [7] Ambler, Scott, “Agile Project Planning Tips,” 2005.
<http://www.ambysoft.com/essays/agileProjectPlanning.html>